Spring

Bean

> *bean*是单例的

singletion 默认单例

可以使用prototype 来多例

单例由spring容器管理，多例不管理

> *bean*不是线程安全的

并没有可变的状态　（不能修改的类）

某种程度上线程安全

但是可变的成员变量要考虑线程安全

> 生命周期

把xml信息传递到beandefinition

构造函数 实例化bean

——》依赖注入

--》Aware接口　-----(beannameaware,beanfactoryaware,applicationaware) 三个要记住

--->BeanPostPrecessor #before

--->初始化方法---(initializingbean，自定义化方法)

--->BeanPostProcessro#after （---->AOP-->动态代理） 这里可以功能增强

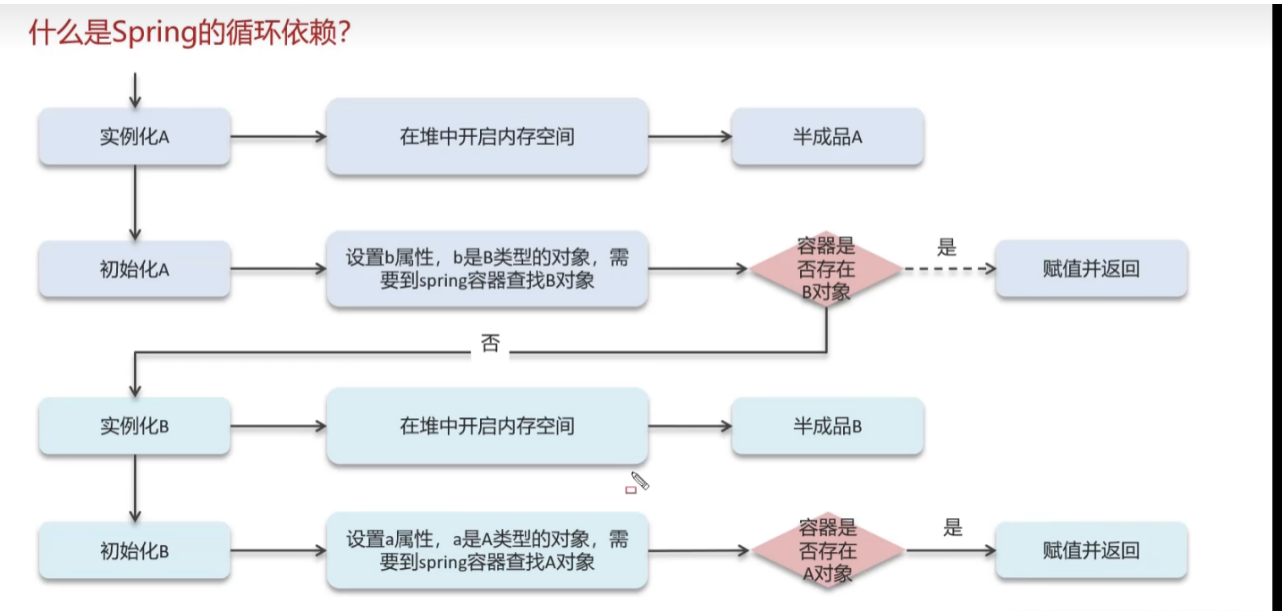
--->销毁bean

> 循环引用

A用到B B用到A

可能会死循环

初始化的循环依赖



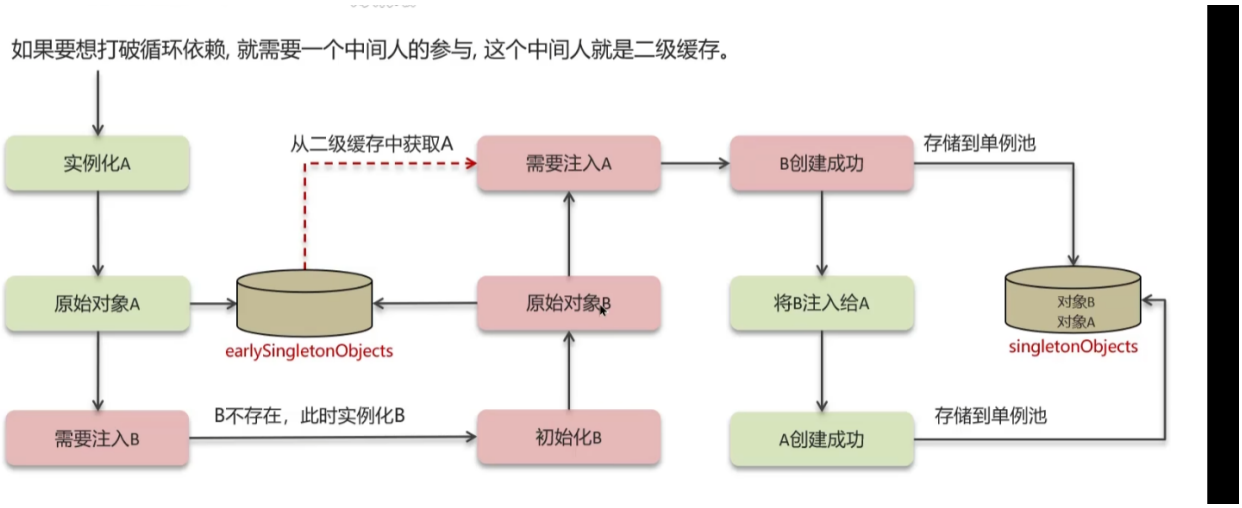
三级缓存解决循环依赖 (都是concurrenthashmap，三级是普通hashmap)

一级缓存 singletonObjects 单例池，缓存已经经历了完整的生命周期，已经初始化完成的bean对象 （为了实现单例模式）

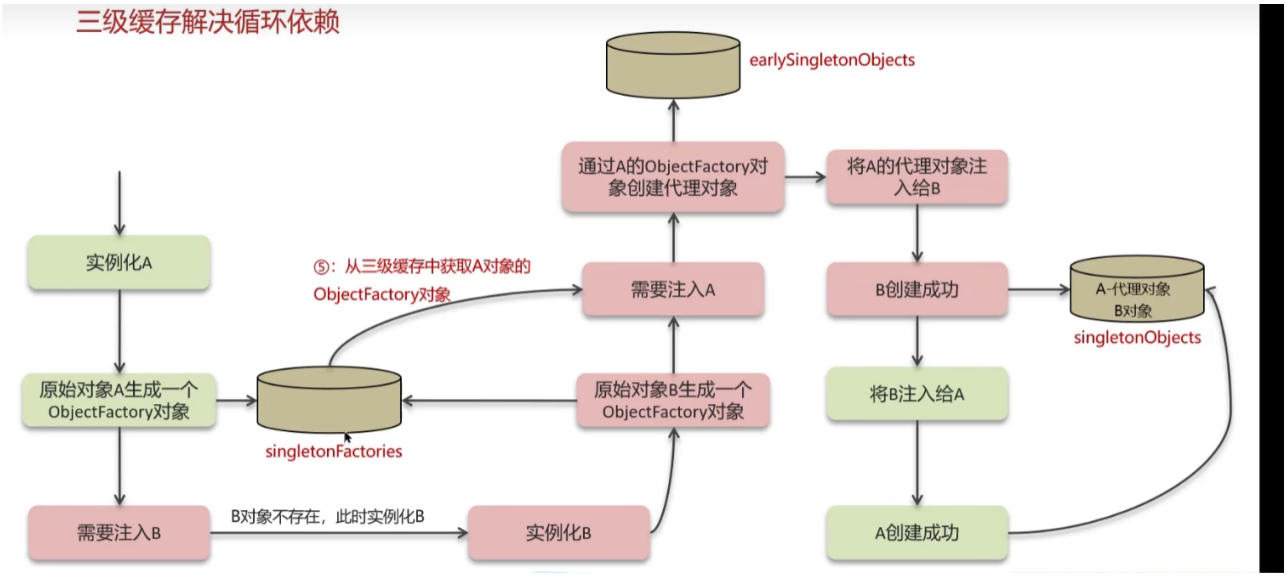
二级缓存 earlySingletionObjects 缓存早期的bean对象（生命周期还没走完）（半成品）

三级缓存 singletonFactories 缓存的是ObjectFactory,表示对象工厂，用来创建某个对象的

二级缓存解决普通循环问题



如果 A是代理对象 那就需要三级缓存解决



对比二级缓存解决 其实就是存了个单例的工厂 然后用工厂拿到A 再把A放到早期单例池

例如事务就要代理对象 @transaction

如果构造方法出了循环依赖 那就用懒加载 @Lazy

AOP

切面

基于动态代理

@Around　是完全控制（代理）

，通常需要先使用 `@Pointcut` 注解来声明一个切点函数，然后再通过其他通知注解（如 `@Before`、`@After`、`@Around` 等）来引用这个切点函数。

```
@Pointcut("execution(* com.example.service.UserService.*(..))")
public void logPointcut() {}

@Before("logPointcut()")  // 引用刚才定义的切点函数
```

事务的本质是AOP　开启事务再执行　失败就回滚

事务及事务失效的场景

使用@Transactional即可使用事务

还能使用 `TransactionTemplate`

```
@Service
public class UserService {

    @Autowired
    private TransactionTemplate transactionTemplate;

    public void updateUser(User user) {
        transactionTemplate.execute(status -> {
            // 业务逻辑
            userMapper.update(user);

            // 如果出现异常，事务会自动回滚,亦可自己catch然后手动回滚
            return null;
```

```
        });
    }
}
```

底层是AOP（动态代理），拦截器管“什么时候做、按什么规则做”，管理器管“具体怎么做”，二者配合实现注解式事务的自动控制。

拦截器基于 AOP 拦截带 `@Transactional` 的方法，先解析注解配置（如传播行为、回滚规则），再根据当前事务状态决定是否开启、暂停事务，最后在方法执行后触发提交或回滚；而事务管理器是实际执行者，比如 `DataSourceTransactionManager` 会操作数据库连接（设置自动提交为 false、调用 commit/rollback），不同数据访问技术对应不同管理器（不继续深入了）

事务注解的两大参数，一个是传播行为，一个是隔离级别（Mysql八股），默认即可

传播行为（Transaction Propagation）是 Spring 事务管理的核心概念，用来定义当一个事务方法调用另一个事务方法时，两个事务如何协同工作（比如是否共用一个事务、是否新建事务等）。

3 个最常用的：

1. **REQUIRED**（默认）：如果当前已有事务，就加入该事务；如果没有，就新建一个事务。比如 ServiceA 调用 ServiceB，两者会在同一个事务里，要么一起提交，要么一起回滚。
2. **REQUIRES_NEW**：无论当前有没有事务，都新建一个独立事务，原事务会被暂停，直到新事务完成。比如 ServiceA 调用 ServiceB 时，B 抛异常回滚，不影响 A 的事务。
3. **SUPPORTS**：如果当前有事务，就加入；如果没有，就以非事务方式执行（不开启新事务）。也就是说，该模式是否支持事务，看调用它的方法是否有事务支持。

失效情况：

情况1：异常捕获处理

让try catch给捕获 但是没有throw 导致spring没处理和回滚

情况2：抛出检查异常

在方法上定义了throws IO异常

Spring默认回滚非检查异常(Runtime)

设置@Transactional(rollback=Exeption.class) 直接顶格 处理所有异常（实现exeption的异常）

情况三：非public方法导致的失效

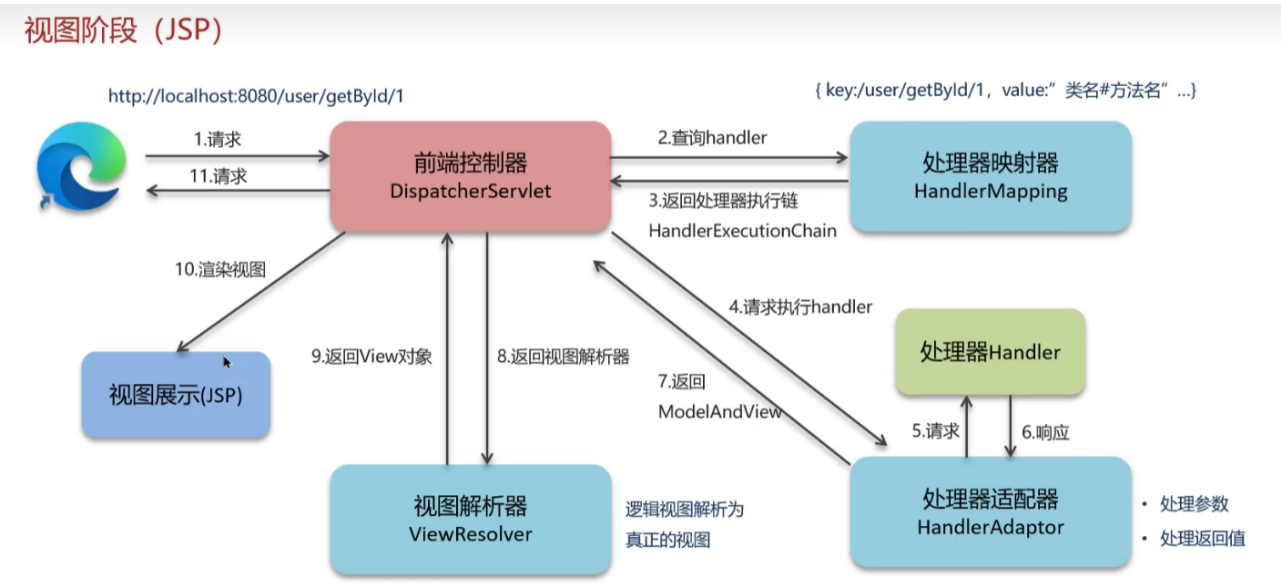
spring默认处理public 与代理有关

java默认是default 方法具有包私有访问权限，只有同一包中的类能够访问 不能被包外的类访问。

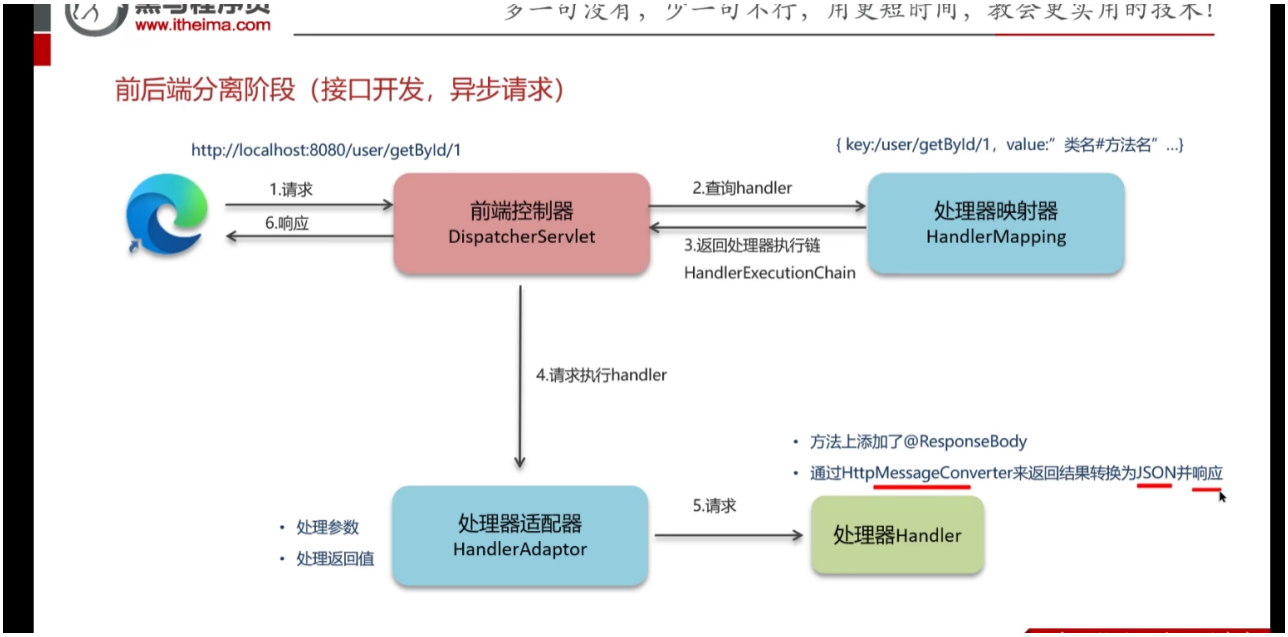
SpringMVC

> 执行流程

视图阶段



这一部分看**PDF**



其实就是把后面的视图省掉了

自动配置原理

首先简单讲，以redis为例，只要在pom里面写了依赖，就会自动注入Bean，直接用RedisTemplate这个Bean。

不需要再写 `@Configuration` 去注册这些常规组件Bean。

```
// 数据库配置类
@Configuration
public class DataSourceConfig {

    // 从配置文件（如 application.properties）读取参数
    @Value("${spring.datasource.url}")
    private String url;
    @Value("${spring.datasource.username}")
    private String username;
    @Value("${spring.datasource.password}")
    private String password;

    // 定义数据源 Bean（比如 HikariCP 连接池）
```

```
    @Bean
    public DataSource dataSource() {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl(url);
        config.setUsername(username);
        config.setPassword(password);
        // 其他配置：超时时间、最大连接数等
        config.setMaximumPoolSize(10);
        return new HikariDataSource(config); // 返回连接池实例
    }

    // 依赖数据源的 JdbcTemplate Bean
    @Bean
    public JdbcTemplate jdbcTemplate() {
        // 直接调用上面定义的 dataSource() 方法，Spring 会自动注入依赖
        return new JdbcTemplate(dataSource());
    }
}
```

@SpringBootApplication包含以下三个注解

@SpringBootConfiguration

@EnableAutoConfiguration （核心）

@ComponentScan ： 自动扫描指定包下的注解Bean，比如@Controller

> ***@EnableAutoConfiguration***

```java
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@AutoConfigurationPackage
@Import({AutoConfigurationImportSelector.class})
public @interface EnableAutoConfiguration {
    no usages
    String ENABLED_OVERRIDE_PROPERTY = "spring.boot.enableautoconfiguration";

    Class<?>[] exclude() default {};

    no usages
    String[] excludeName() default {};
}
```

@AutoConfigurationPackage:指定自动配置的基础包路径,当前注解标注的类所在的包，及其子包下的组件，需要被纳入自动扫描和管理范围

`@Import(AutoConfigurationImportSelector.class)` 导入了 `AutoConfigurationImportSelector` 类，这个类负责从所有自动配置类中挑选出与当前应用环境匹配的配置。

1、读取`META-INF/spring.factories`文件，查找所有实现了 `AutoConfiguration` 接口的类，加载所有候选自动配置类（如RedisAutoConfiguration）。

2、通过@Conditional注解筛选生效配置（如@ConditionalOnClass检查依赖是否存在）。

> *@Conditional*

```java
@Configuration
@ConditionalOnClass(DataSource.class)
public class DataSourceAutoConfiguration {
    @Bean
    @ConditionalOnMissingBean
    public DataSource dataSource() { ... }
}
//当类路径有 DataSource 且容器里没有自定义 DataSource 时，才自动配置默认的数据源。
```

Spinrg三件套常见注解

> *autowired和resource的注解*

autowired是spring的实现，resource是j2ee的实现

autowired优先按类型查找

找不到再按名称查

可以结合@Qualifier 来解决接口多实现问题

resource优先按名称

按照变量名 来查找，如果找不到就再按类型找，再找不到（多实现）的话，可以指定 `name` 属性（最直接）

```
@Component
class MyServiceImpl implements MyService {}
默认规则：类名首字母小写

类名 MyServiceImpl → bean 名称 "myServiceImpl"

所以在容器里实际注册的名字就是 "myServiceImpl"

    @Resource
    private MyService myServiceImpl; 成功注入

    @Resource(name="myServiceImpl")
private MyService myService; 成功注入
```

> *Spring*

自己背吧

注解	说明
@Component、@Controller、@Service、@Repository	使用在类上用于实例化Bean
@Autowired	使用在字段上用于根据类型依赖注入
@Qualifier	结合@Autowired一起使用用于根据名称进行依赖注入
@Scope	标注Bean的作用范围
@Configuration	指定当前类是一个 Spring 配置类，当创建容器时会从该类上加载注解
@ComponentScan	用于指定 Spring 在初始化容器时要扫描的包
@Bean	使用在方法上，标注将该方法的返回值存储到Spring容器中
@Import	使用@Import导入的类会被Spring加载到IOC容器中
@Aspect、@Before、@After、@Around、@Pointcut	用于切面编程（AOP）

> *SpringMVC*

注解	说明
@RequestMapping	用于映射请求路径，可以定义在类上和方法上。用于类上，则表示类中的所有的方法都是以该地址作为父路
@RequestBody	注解实现接收http请求的json数据，将json转换为java对象
@RequestParam	指定请求参数的名称
@PathViriable	从请求路径下中获取请求参数(/user/{id})，传递给方法的形式参数
@ResponseBody	注解实现将controller方法返回对象转化为json对象响应给客户端
@RequestHeader	获取指定的请求头数据
@RestController	@Controller + @ResponseBody

> *Springboot*

注解	说明
@SpringBootConfiguration	组合了- @Configuration注解，实现配置文件的功能
@EnableAutoConfiguration	打开自动配置的功能，也可以关闭某个自动配置的选
@ComponentScan	Spring组件扫描

Mybatis

> 执行流程

mybatis-config.xml

配置

指定mapper

有springboot就不用这些了

```
public interface UserMapper {
    User selectById(@Param("id") Long id);

     // 返回多条记录
    List<User> selectByStatus(@Param("status") String status);
}



<!-- resources/mapper/UserMapper.xml -->
<mapper namespace="com.example.mapper.UserMapper">
    <select id="selectById" parameterType="long"
resultType="com.example.model.User">
        SELECT * FROM user WHERE id = #{id}
    </select>

         <select id="selectByStatus" parameterType="string"
resultType="com.example.model.User">
        SELECT * FROM user WHERE status = #{status}
    </select>
</mapper>

        //#{status}和@Param("status")是要对上的
        多个参数时必须用 @Param，单参数情况下不加 @Param 可能也行
```

构建会话工厂---》创建会话SqlSession 项目与sql的会话----》执行器(里面有映射信息)---》MappedStatement对象 ----->数据库

> 延迟加载

默认不开启

什么是延迟加载？

简单讲就是按需加载 fetchtype可以设置为lazy （局部修改）

全局修改:lazyloadingenabled

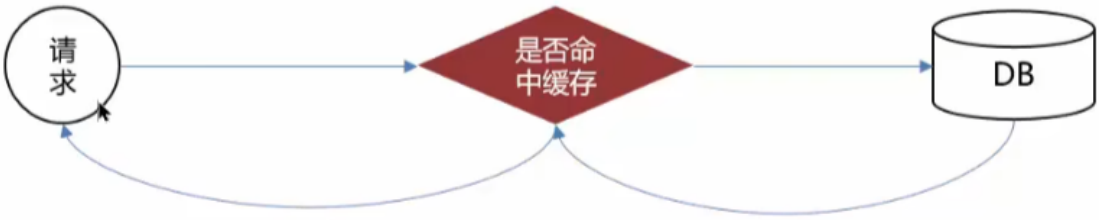
用的cglib代理对象

通常场景是 一个对象里面包着另外一个对象，此时另外一个对象可以延迟加载，此时这个对象被代理，只有在get里面的对象时才会执行查询（代理拦截和invoke）

延迟加载的底层原理知道吗?

1. 使用CGLIB创建目标对象的代理对象
2. 当调用目标方法时，进入拦截器invoke方法，发现目标方法是null值，执行sql查询
3. 获取数据以后，调用set方法设置属性值，再继续查询目标方法，就有值了

> 缓存



- 本地缓存，基于PerpetualCache，本质是一个HashMap
- 一级缓存：作用域是session级别
- 二级缓存：作用域是namespace和mapper的作用域，不依赖于session

一级缓存：单一sqlsession

二级缓存：不同会话之间的

二级缓存默认关闭 可以开启 要记得打上标签

一级缓存和二级缓存namespeaces增删改后 二级缓存select中的缓存会被清空

只有会话提交或者关闭以后 一级缓存中的数据才会存到二级缓存

> *#和$的区别？*

#会替换为占位符"?"，也就是PreaparedStatment的防注入方式。

$不能防注入

> *JDBC的回忆*

Connection建立，调用Connection的create方法创建statement，调用statement的方法执行SQL语句，然后close资源

记得要用try with resources和使用事务，还有PreparedStatement

```
import java.sql.*;

public class JdbcTransactionDemo {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/testdb";
        String username = "root";
        String password = "123456";

        Connection conn = null;
        PreparedStatement ps = null;

        try {
            // 1. 获取连接
            conn = DriverManager.getConnection(url, username,
password);

            // 2. 关闭自动提交
            conn.setAutoCommit(false);
```

```
            // 3. 准备 SQL
            String sql1 = "INSERT INTO user(id, name) VALUES (?,
?)";
            ps = conn.prepareStatement(sql1);
            ps.setLong(1, 1L);
            ps.setString(2, "Alice");
            ps.executeUpdate();


            // 4. 提交事务
            conn.commit();
            System.out.println("事务提交成功！");
        } catch (SQLException e) {
            e.printStackTrace();
            try {
                if (conn != null) {
                    // 5. 回滚事务
                    conn.rollback();
                    System.out.println("事务回滚！");
                }
            } catch (SQLException ex) {
                ex.printStackTrace();
            }
        } finally {
            // 6. 关闭资源
            try {
                if (ps != null) ps.close();
                if (conn != null) conn.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Spring代理相关

Spring 默认优先使用 **JDK** 动态代理，但 如果目标类没有实现接口，Spring 只能使用 **CGLIB**。其核心原因在于 **JDK** 动态代理的限制 和 **CGLIB** 的继承机制。

CGLIB 代理的本质是 继承目标类并重写方法，但 `final` 方法不能被重写，所以当 CGLIB 试图对 `final` 方法进行代理增强时，可能会导致 代理对象内部依赖的 **Spring** 组件未被正确初始化，从而引发 空指针异常（**NullPointerException, NPE**）。

CGLIB 代理是通过 创建目标类的子类 来实现的，但 `final` 方法不能被子类重写，所以当外部调用该 `final` 方法时，实际上调用的是 目标类（父类）自身的方法，而代理对象的依赖注入（**DI**）在 **Spring** 中是针对代理类进行的，目标类本身并没有被 **Spring** 进行 **DI**，因此会导致 `null` 引用，从而引发 空指针异常。

spring一些杂

依赖注入

Spring直接autowired即可

其他方式如下：

> 构造器注入

```
@Service
public class UserService {

    private final UserMapper userMapper;

    // 构造器注入
    public UserService(UserMapper userMapper) {
```

```
        this.userMapper = userMapper;
    }
}
//Spring 容器创建 UserService 时：

发现 UserService 只有一个构造器

找到类型为 UserMapper 的 Bean

调用构造器 new UserService(userMapper)

注入完成

☑ 整个过程自动完成，不需要手动调用构造器
```

Spring 容器创建 Bean 时，如果找不到依赖 Bean，会报错，防止创建了一个包含空对象的 Bean，防止使用时出现 `NullPointerException`

使用 `final` 修饰依赖，保证 Bean 生命周期内不会被修改，提高对象设计的健壮性

> *setter注入*

```
@Service
public class UserService {

    private UserMapper userMapper;

    // Setter 注入
    @Autowired
    public void setUserMapper(UserMapper userMapper) {
        this.userMapper = userMapper;
    }

    public void updateUser(User user) {
        userMapper.update(user);
    }
}
setter方法不能动final的变量，但构造对象（构造函数）的时候可以
```

比构造器糟糕，废物。

spring security

> 怎么保证安全的

核心就是先认证再授权，还带防攻击

认证的时候，它会通过 AuthenticationManager 之类的组件校验你的身份，比如用户名密码对不对，密码都是用 BCrypt 这种加密算法存的，明文根本不会落地。要是用 JWT 令牌登录，它还能验证令牌的签名，防止篡改。

授权这块更细，能按角色（比如 ADMIN）或者具体权限（比如删用户）来限制，接口级别的用 SecurityFilterChain 配置哪些路径谁能访问，方法级别的直接加 @PreAuthorize 注解，比如“只有管理员才能调这个方法”。

默认就防 CSRF 攻击，非 GET 请求得带令牌；还会加各种安全响应头，防 XSS